

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

1. AUTHENTICATION BYPASS VULNERABILITIES

Response Manipulation / Status Code Bypass

Critical

Description: Manipulating server responses to trick the client-side logic into believing authentication succeeded, even when credentials were invalid.

Steps to Reproduce:

- Intercept login request with Burp Suite / Proxyman
- Enter invalid credentials and submit
- Capture the 401/403 error response
- Change status code to 200 OK and modify response body to include valid session token or success message
- Forward the modified response to the browser
- Observe if the application grants access without valid credentials

Impact:

Complete account takeover on any user account without knowing credentials. Can bypass MFA if the client relies on response validation.

Tools:

Burp Suite, Proxyman, OWASP ZAP, Browser DevTools

References:

- ZACK0X01 - Account Takeover Tutorial 2026
- Twitter/X bug bounty threads
- Medium: infosecwriteups.com

JWT Algorithm Confusion (RS256 to HS256)

Critical

Description: When JWT validation does not specify the algorithm, attackers can change alg header from RS256 to HS256 and sign with the public key as HMAC secret.

Steps to Reproduce:

- Obtain the JWT token from login response
- Decode the JWT header and change "alg": "RS256" to "alg": "HS256"
- Obtain the public key from /.well-known/jwks.json or certificate
- Use the public key as HMAC secret to sign modified payload (e.g., change user role to admin)
- Send the forged token with requests to protected endpoints
- Verify if server accepts the token and grants elevated privileges

Impact:

Privilege escalation to admin, account takeover, unauthorized data access across all user accounts.

Tools:

jwt_tool, Burp Suite JWT Editor, Python (PyJWT), openssl

References:

- PortSwigger Web Security Academy
- Intigriti BugQuest 2026 - Day 24
- Medium: JWT attacks 2026

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

OAuth Redirect URI Manipulation

High

Description: Exploiting weak redirect_uri validation in OAuth flows to steal authorization codes or tokens by redirecting to attacker-controlled domains.

Steps to Reproduce:

- Identify OAuth endpoints (/oauth/authorize, /auth, etc.)
- Test redirect_uri parameter with variations: attacker.com, target.com.attacker.com, javascript:alert(1)
- Check if the server validates exact match vs. partial match
- If partial match allowed, register attacker.com subdomain containing target domain name
- Construct phishing link with malicious redirect_uri
- When victim clicks, authorization code is sent to attacker server
- Exchange code for access token using client_id/client_secret (if public client)

Impact:

Account takeover via stolen tokens, phishing attacks, unauthorized access to user data through compromised OAuth flow.

Tools:

Burp Suite, OAuthScan, custom Python scripts

References:

- PortSwigger OAuth labs
- Twitter: @filedescriptor OAuth threads
- Medium: OAuth security 2026

JavaScript postMessage Origin Validation Bypass

High

Description: Exploiting weak origin validation in postMessage handlers used for authentication (e.g., "Sign in with X" features).

Steps to Reproduce:

- Find postMessage listeners in JavaScript: search for window.addEventListener("message"
- Analyze origin validation logic (e.g., search(), indexOf(), regex)
- If using .search() or .indexOf(), test if attacker domain contains target string
- Example: target.com validation might pass for attacker-target.com or target.com.attacker.com
- Host malicious page that sends crafted postMessage to target iframe
- If origin check passes, authentication credentials/tokens are leaked to attacker

Impact:

Complete account takeover on platforms using social login. Affected Twitter Digits, major platforms.

Tools:

Browser DevTools, Burp Suite, custom HTML/JS

References:

- Medium: filedescriptor - JavaScript dot bypass
- Twitter/X security research threads
- Bug bounty writeups 2025-2026

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

2. SESSION MANAGEMENT VULNERABILITIES

Session Fixation via Login

High

Description: When a session ID is set before authentication and remains the same after login, allowing attackers to hijack authenticated sessions.

Steps to Reproduce:

- Visit login page and capture the pre-authentication session cookie (e.g., PHPSESSID=abc123)
- Do NOT log in yet
- Send the pre-auth session link to victim or set victim cookie to this value
- Victim logs in with the fixed session ID
- Attacker uses the same session ID to access victim account
- Check if session ID changes after successful authentication

Impact:

Session hijacking, account takeover without knowing credentials. Particularly dangerous on shared computers.

Tools:

Burp Suite, Browser DevTools, cookie editor extensions

References:

- OWASP Testing Guide
- Medium: session management 2026
- LinkedIn: AppSec articles

Weak Session Token Predictability

Medium

Description: Session tokens that are sequential, time-based, or use weak randomness allowing attackers to guess valid sessions.

Steps to Reproduce:

- Collect multiple session tokens by logging in/out repeatedly
- Analyze token structure (length, character set, patterns)
- Check for sequential patterns: sess_1001, sess_1002, sess_1003
- Check for timestamps or user ID embedded in token
- Use Burp Sequencer to test entropy
- If predictable, brute-force or calculate valid tokens for other users

Impact:

Mass account takeover via session enumeration, unauthorized access to random user accounts.

Tools:

Burp Sequencer, Python (randomness analysis), Entropy analysis tools

References:

- PortSwigger Session Management labs
- Medium: token prediction attacks
- Twitter: @zseano tips

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

Session Not Invalidated on Logout

Medium

Description: When clicking logout does not actually destroy the server-side session, allowing replay of old session tokens.

Steps to Reproduce:

- Log in and capture the session cookie/token
- Perform sensitive action (e.g., view profile) to confirm session works
- Click logout and capture the logout request/response
- Immediately replay the pre-logout request with the old session token
- If the server still responds with user data, session was not invalidated
- Also test: logout from one browser, use token in another browser/incognito

Impact:

Session hijacking even after user believes they logged out. Physical access attacks, shared device risks.

Tools:

Burp Repeater, curl, Browser DevTools

References:

- OWASP Testing Guide
- Medium: session invalidation bugs
- LinkedIn security posts

Cross-Subdomain Session Cookie Sharing

Medium

Description: Session cookies set with Domain=.target.com accessible to all subdomains, allowing XSS on any subdomain to steal main domain sessions.

Steps to Reproduce:

- Inspect session cookie attributes: Domain, Path, Secure, HttpOnly, SameSite
- If Domain=.target.com, cookie is sent to all subdomains
- Find XSS or subdomain takeover on any *.target.com subdomain
- Inject JavaScript to document.cookie and exfiltrate session cookie
- Use stolen cookie on main domain to impersonate victim
- Check if SameSite=None or missing (allows cross-site requests)

Impact:

Session theft via XSS on less secure subdomains, account takeover, bypassing main domain security controls.

Tools:

Browser DevTools, subdomain enumeration tools, XSS payloads

References:

- Twitter: @albinowax SameSite research
- Medium: cookie security 2026
- LinkedIn: web security articles

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

3. PASSWORD & CREDENTIAL VULNERABILITIES

Username Enumeration via Error Messages

Medium

Description: Different error messages for valid vs invalid usernames allowing attackers to enumerate registered accounts.

Steps to Reproduce:

- Submit login with valid username but wrong password
- Note the exact error message (e.g., "Invalid password")
- Submit login with completely random username
- Note the error message (e.g., "User not found")
- If messages differ, enumerate usernames via automated requests
- Also test: response timing differences (valid users may take longer due to password hashing)

Impact:

Targeted password spraying, credential stuffing, social engineering, account enumeration for phishing campaigns.

Tools:

Burp Intruder, ffuf, custom Python scripts

References:

- OWASP Testing Guide
- Medium: user enumeration techniques
- Twitter: bug bounty tips 2026

Password Reset Token Predictability / Leakage

Critical

Description: Weak reset tokens (short, sequential, time-based) or tokens leaked in Referer headers, email clients, or third-party analytics.

Steps to Reproduce:

- Request password reset for your own account
- Capture reset token from email URL
- Analyze token: length, entropy, structure (e.g., 6-digit code vs UUID)
- If short numeric: brute-force using Burp Intruder (000000-999999)
- Check if token is leaked in Referer header when clicking email links
- Check if reset link HTTP request includes token in query params (logged by analytics)
- Test if old reset tokens remain valid after requesting new one

Impact:

Complete account takeover on any user. Mass account compromise if token is guessable.

Tools:

Burp Suite, Python (token analysis), email client inspection

References:

- Medium: password reset vulnerabilities 2026
- Twitter: @hacker_account takeover threads
- Intigriti BugQuest

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

Password Reset via Host Header Poisoning

High

Description: When password reset emails use the Host header to construct reset links, attackers can poison the header to receive victim reset tokens.

Steps to Reproduce:

- Intercept password reset request in Burp Suite
- Modify Host header to attacker-controlled domain: Host: attacker.com
- X-Forwarded-Host may also work if behind proxy
- Forward the request
- If victim receives email with reset link pointing to attacker.com/token
- Attacker captures token from access logs when victim clicks
- Use token to reset victim password

Impact:

Account takeover on any user who requests password reset. No interaction needed with victim beyond normal reset flow.

Tools:

Burp Suite, attacker server with access logs

References:

- PortSwigger Host Header labs
- Medium: password reset poisoning
- Twitter: web security threads

No Rate Limiting on Login / Brute Force

Medium

Description: Missing rate limiting allows unlimited login attempts, enabling brute-force and password spraying attacks.

Steps to Reproduce:

- Send 10-20 login requests with wrong password in rapid succession
- Check if account gets locked or IP gets blocked
- If no lockout, automate with Burp Intruder or Hydra
- Test password spraying: 1 common password across many usernames
- Check for CAPTCHA bypass (client-side only, reusable tokens)
- Test distributed attack: same credentials from different IPs
- Check if rate limit resets immediately or after short delay

Impact:

Credential stuffing, brute-force attacks, mass account compromise using leaked password databases.

Tools:

Burp Intruder, Hydra, ffuf, custom scripts

References:

- OWASP Testing Guide
- Medium: rate limiting bypass 2026
- LinkedIn: AppSec discussions

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

4. MULTI-FACTOR AUTHENTICATION (MFA) BYPASS

MFA Bypass via Response Manipulation

Critical

Description: When MFA verification is enforced client-side or response indicates success/failure without proper server validation.

Steps to Reproduce:

- Log in with valid username/password
- When prompted for MFA code, enter any random code
- Intercept the MFA verification request/response
- If response contains JSON like {"success": false}, change to {"success": true}
- If response contains redirect URL to /mfa-failed, change to /dashboard
- Forward modified response
- Check if application grants access without valid MFA code

Impact:

Complete MFA bypass, account takeover even on accounts with 2FA/SMS/OTP enabled.

Tools:

Burp Suite, Proxyman, OWASP ZAP

References:

- Medium: [MFA bypass techniques 2026](#)
- Twitter: [@zack0x01 threads](#)
- Bug bounty writeups

OTP Brute Force (No Rate Limiting)

High

Description: 6-digit OTP codes with no rate limiting can be brute-forced within hours (1,000,000 combinations but often shorter timeout).

Steps to Reproduce:

- Initiate login and request OTP/SMS code
- Enter random 6-digit code and intercept request
- Send to Burp Intruder with payload range 000000-999999
- Set thread count high and observe responses
- Look for response length/status code difference on correct code
- If rate limited after X attempts, test parallel sessions or IP rotation
- Check if OTP remains valid for extended period (should be <5-10 minutes)

Impact:

Account takeover on any account with SMS/Email OTP. Particularly dangerous for banking/financial apps.

Tools:

Burp Intruder, ffuf, Python scripts

References:

- Medium: [OTP brute force 2026](#)
- Twitter: [bug bounty OTP threads](#)
- LinkedIn: [security research](#)

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

MFA Bypass via CSRF on Disable MFA

High

Description: Missing CSRF protection on MFA disable/backup code generation endpoints allows attackers to disable victim MFA via crafted links.

Steps to Reproduce:

- Log in and navigate to MFA settings
- Capture request to disable MFA or generate backup codes
- Check for CSRF token in request
- If missing, craft HTML form that submits to /disable-mfa
- Trick logged-in victim into visiting attacker page
- If MFA gets disabled without confirmation, CSRF successful
- Attacker can now login with just password (no MFA)

Impact:

MFA removal on victim accounts, reducing security to single-factor, enabling subsequent credential-based takeover.

Tools:

Burp Suite, custom HTML/JS

References:

- PortSwigger CSRF labs
- Medium: MFA CSRF 2026
- Twitter: AppSec threads

Backup Code Reuse / Predictability

Medium

Description: MFA backup codes that can be reused multiple times or are generated with weak entropy allowing prediction.

Steps to Reproduce:

- Enable MFA and generate backup codes
- Use one backup code to log in
- Try using the SAME backup code again
- If it works again, codes are not single-use
- Analyze backup code format (e.g., 8-digit numbers)
- If short/sequential, brute-force backup codes when MFA device is lost
- Check if backup codes are stored in plaintext or weakly hashed

Impact:

Permanent MFA bypass using leaked/reused backup codes, account takeover even after MFA is re-enabled.

Tools:

Manual testing, Burp Suite

References:

- Medium: backup code vulnerabilities
- Twitter: MFA security threads
- LinkedIn: security articles

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

5. BUSINESS LOGIC & ACCESS CONTROL (IDOR)

IDOR in Login-Protected Endpoints

Critical

Description: Accessing other users data by manipulating object references in URLs/parameters after authentication.

Steps to Reproduce:

- Log in as User A
- Navigate to profile/settings and note URL pattern (e.g., /api/user/1234)
- Capture the request in Burp Suite
- Change user ID from 1234 to 1235, 1236, etc.
- If you see User B data, IDOR exists
- Test on all endpoints: /orders?id=X, /invoices?user=X, /messages?thread=X
- Also test with UUIDs: try other users UUIDs from public sources

Impact:

Data breach of all user accounts, PII exposure, financial data theft, horizontal privilege escalation.

Tools:

Burp Suite, Autorize extension, Firefox Multi-Account Containers

References:

- *Intigriti BugQuest 2026*
- *Medium: IDOR hunting guide*
- *Twitter: @zseano IDOR threads*

Mass Assignment on Registration/Profile Update

High

Description: When registration or profile update endpoints accept additional parameters not shown in UI (e.g., role, is_admin, credits).

Steps to Reproduce:

- Capture registration request: POST /api/register
- Add extra parameters to JSON body: "role": "admin", "is_verified": true
- Submit and check if account is created with elevated privileges
- Test profile update: add "balance": 10000 or "subscription": "premium"
- Compare schema with API documentation or JavaScript source code
- Check if server filters unknown parameters or blindly saves all

Impact:

Privilege escalation to admin, free premium access, unauthorized credit/balance manipulation.

Tools:

Burp Suite, Postman, browser DevTools

References:

- *Medium: mass assignment 2026*
- *Twitter: API security threads*
- *LinkedIn: AppSec articles*

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

GraphQL Authorization Bypass

High

Description: GraphQL resolvers that check authentication but fail to validate authorization on queries/mutations.

Steps to Reproduce:

- Identify GraphQL endpoint (usually /graphql)
- Introspect schema to find sensitive queries: user, orders, adminUsers
- Query user(id: "123") with your token
- Query user(id: "124") - if it returns other user data, authorization missing
- Test mutations: updateUser, deleteUser with other user IDs
- Check if @auth directive is applied to all resolvers or just some

Impact:

Complete data access across all users, admin function abuse, data modification on other accounts.

Tools:

GraphQL Playground, Burp Suite, InQL extension

References:

- *Intigriti BugQuest 2026 - Day 20*
- *Medium: GraphQL security 2026*
- *Twitter: GraphQL bug bounty*

HTTP Method-Based Authorization Bypass

Medium

Description: When authorization checks only apply to specific HTTP methods (e.g., GET protected but POST/PUT bypassed).

Steps to Reproduce:

- Access admin endpoint with GET: GET /admin/users → 403 Forbidden
- Try same endpoint with POST: POST /admin/users → 200 OK?
- Test PUT, DELETE, PATCH on restricted endpoints
- Check if OPTIONS request reveals allowed methods
- If POST bypasses auth, try to perform admin actions via POST
- Also test: add X-HTTP-Method-Override: PUT header to GET request

Impact:

Privilege escalation, unauthorized admin actions, data manipulation via bypassed access controls.

Tools:

Burp Repeater, curl, custom scripts

References:

- *Intigriti BugQuest 2026 - Day 15*
- *Medium: method bypass techniques*
- *Twitter: access control threads*

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

6. NEW & ADVANCED TECHNIQUES (2026)

JavaScript Source Code Analysis for Hidden Endpoints

Modern SPAs and login flows load large JS bundles containing API endpoints, internal paths, and sometimes hardcoded credentials. Use sourcemap extraction and regex searches for patterns like /api/, /admin/, /internal/, apiKey, token, password.

Tools: SourceMap extractor, JSMiner, Burp JS Link Finder, `grep -E "(api|admin|internal|token|key)"`

Race Condition in Login/Registration

Sending multiple simultaneous requests to exploit time-of-check to time-of-use (TOCTOU) vulnerabilities. Example: registering same username multiple times simultaneously to create duplicate accounts or bypassing email verification by confirming multiple times.

Tools: Turbo Intruder (Burp extension), Python threading, parallel curl requests

Second-Order SQL Injection via Login

When user input is stored during registration but executed during login. Example: registering with username containing SQL payload that executes when the user later logs in and the application retrieves profile data.

Payload: `username: admin' UNION SELECT * FROM passwords--`

Tools: SQLMap (second-order mode), manual testing

SSRF via OAuth / Login Callbacks

OAuth callbacks, SAML endpoints, and login redirects that fetch internal resources. Manipulating callback URLs to make the server request internal services (metadata, EC2, Kubernetes).

Payload: `redirect_uri=http://169.254.169.254/latest/meta-data/`

Tools: Burp Collaborator, SSRFmap

Web Cache Poisoning on Login Pages

Poisoning web caches by sending crafted requests that cause the cache to serve attacker-controlled content to other users visiting the login page.

Tools: Burp Suite Web Cache Poisoning extension, Param Miner, manual header fuzzing

DOM-Based XSS in Login Flows

XSS that occurs entirely in browser via client-side JavaScript processing of login callbacks, OAuth tokens, or URL fragments. Often missed by server-side scanners.

Look for: `location.hash`, `window.name`, `postMessage`, `localStorage` reads in login JS

Tools: DOM Invader (Burp), browser DevTools

API Key / Token Extraction from Mobile Login

Mobile apps often embed API keys in APK/IPA files or use weak certificate pinning. Reverse engineering mobile login flows to extract hardcoded credentials or bypass SSL pinning to intercept traffic.

Tools: Frida, Objection, APKTool, JADX, Burp Suite Mobile Assistant

SAML Authentication Bypass

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

XML Signature Wrapping (XSW) attacks on SAML assertions, allowing attackers to modify identity assertions while keeping signature valid.

Tools: SAML Raider (Burp extension), SAMLTracer (Firefox), manual XML manipulation

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

7. RECONNAISSANCE METHODOLOGY FOR LOGIN TARGETS

Subdomain Enumeration for Login Panels

Use multiple tools to find login panels on subdomains: admin.target.com, portal.target.com, vpn.target.com, mail.target.com. Visual recon with Aquatone/GoWitness for quick identification.

```
Commands:
subfinder -d target.com | httpprobe
assetfinder --subs-only target.com | anew subs.txt
aquatone -out screenshots/
```

Google Dorking for Login Pages

```
site:target.com inurl:login
site:target.com inurl:admin
site:target.com intitle:"login"
site:target.com ext:php inurl:auth
site:target.com inurl:oauth
site:target.com inurl:saml
```

Wayback Machine for Old Login Endpoints

Historical URLs often reveal deprecated login endpoints, old API versions, and forgotten authentication mechanisms.

```
Commands:
waybackurls target.com | grep -E "(login|auth|oauth|api)" | anew old_endpoints.txt
```

GitHub Recon for Login Secrets

Developers commit API keys, OAuth secrets, test credentials, and internal login URLs to public repositories.

```
Queries:
"target.com" "password"
"target.com" "api_key"
"target.com" "client_secret"
"target.com" "oauth_token"
```

8. ESSENTIAL TOOLS CHEAT SHEET

Proxy & Interception

- Burp Suite Pro/Community - Full proxy, repeater, intruder, extensions
- OWASP ZAP - Open source alternative to Burp
- Proxyman - macOS-focused proxy with modern UI
- Fiddler - Windows proxy, good for .NET apps

Authentication Testing

- JWT Editor (Burp extension) - JWT manipulation and signing
- SAML Raider (Burp extension) - SAML testing
- OAuthScan - OAuth flow vulnerability scanner
- Autorize (Burp extension) - Automated authorization testing

Reconnaissance

- Subfinder / Amass / Assetfinder - Subdomain enumeration
- httpx / httpprobe - Fast HTTP probing
- Aquatone / GoWitness - Visual reconnaissance
- Waybackurls / gau - Historical URL discovery
- Nuclei - Template-based vulnerability scanning

Brute Force & Fuzzing

- ffuf / wfuzz - Fast web fuzzer
- Hydra - Network login cracker
- Burp Intruder - Brute force within proxy workflow
- Turbo Intruder - Fast HTTP attacks (race conditions)

Session Analysis

- Burp Sequencer - Session token entropy analysis
- jwt_tool - JWT testing and cracking
- Cookie Editor - Browser extension for cookie manipulation

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

9. BUG BOUNTY REPORTING BEST PRACTICES

Title Format

[Vulnerability Type] on [Domain] via [Endpoint/Parameter] leading to [Impact]

Example: [IDOR] on api.target.com via /user/{id} leading to PII disclosure of all users

Reproduction Steps

1. Numbered, exact steps
2. Include full URLs
3. Mention tool versions if relevant
4. Attach HTTP request/response in code blocks
5. Screenshots with annotations (arrows, highlights)
6. Video PoC for complex multi-step bugs (keep under 2 minutes)

Impact Statement

- Explain business impact, not just technical
- "An attacker can view all customer order history" vs "IDOR exists"
- Mention data sensitivity (PII, financial, health)
- Chain with other bugs to show maximum impact
- Suggest realistic attack scenario

Severity Self-Triage

Before submitting, ask: "If I were the developer, could I fix this with only this report?"

If no, add more detail.

Reference CVSS 3.1 calculator for consistent severity ratings.

Login Pages Bug Bounty - Comprehensive Bug List & Techniques

Sourced from Twitter, Medium, LinkedIn, Bug Bounty Community | 2026 Edition

END OF DOCUMENT

This document is compiled from public bug bounty research, Twitter/X security community threads, Medium infosec publications, LinkedIn AppSec articles, and official platform resources (HackerOne, Bugcrowd, Intigriti).

Always test within scope and with proper authorization.

Compiled: May 2026